

Reviewer Pack — Algebraic Hodge Theory Bounds for Frege Proof Complexity

Entry d04434ce6aef · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 03:51:32 UTC

Algebraic Hodge Theory Bounds for Frege Proof Complexity

Entry ID: d04434ce6aef **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 03:51:32 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

[‘For any explicit polynomial $f(x)$ in $R[x]$, let $H(f)$ be the Hodge structure of its vanishing cycles modulo 2. For a given n -variable CNF formula F , define the Frege proof complexity as the number of variables that need to be assigned values during the proof process.’, ‘Then, there exists a constant $c > 0$ such that for all n -variable CNF formulas F with Frege proof complexity greater than $n/2$, the Hodge rank of $f(F)$ is at least $c * \log(n)$.’, ‘Conversely, for any polynomial $p(x)$ and an integer $k \leq n/2$, if there exists a CNF formula F such that $f(F) = p(x)$ and the Frege proof complexity of F is less than k , then the Hodge rank of $p(x)$ is at most 2^k .’]

Rationale (proposer’s reasoning):

[‘The algebraic geometry of Hodge theory provides a rich framework for studying algebraic varieties and their properties. By associating Hodge structures with CNF formulas, we can potentially uncover new insights into the complexity of Frege proofs.’, ‘This conjecture leverages the algebraic structure to provide a lower bound on Frege proof complexity, which has not been explored before in this context.’, ‘If true, it would suggest that certain complex properties of Hodge structures are related to computational hardness.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 08d0d04e70b80e0e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for 30 random seeds, at least 80% of the CNF formulas with Frege proof complexity greater than $n/2$ have Hodge ranks that are at least $c * \log(n)$, and no polynomial with a Hodge rank less than or equal to 2^k has a corresponding CNF formula with Frege proof complexity less than k .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge Theory" AND "Frege proof complexity" - "Algebraic Geometry" AND CNF formula AND Hodge structure" - "polynomial Hodge structure" AND Frege AND CNF

Top relevant hits considered: - [<http://arxiv.org/abs/0803.0499v2>] Abelian Hurwitz-Hodge integrals - [<http://arxiv.org/abs/1608.04797v1>] Theta-stratifications, Theta-reductive stacks, and applications - [<http://arxiv.org/abs/2508.17748v1>] Q-factoriality and Hodge-Du Bois theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, complexity):
        # Generate a random n-variable CNF formula with given complexity
        clauses = set()
        for _ in range(complexity):
            clause = []
            for _ in range(random.randint(1, n)):
                var = random.choice(range(1, n + 1))
                if random.choice([True, False]):
                    clause.append(var)
                else:
                    clause.append(-var)
            clauses.add(tuple(sorted(clause)))
        return clauses

    def frege_proof_complexity(cnf):
        # Placeholder for actual Frege proof complexity computation
        return len(cnf) # Simplified for testing purposes

    def polynomial_representation(cnf):
        # Placeholder for actual polynomial representation computation
```

```

n = max(abs(var) for var in cnf)
poly = [0] * (n + 1)
for clause in cnf:
    product = 1
    for var in clause:
        if var > 0:
            product *= (1 - x ** var)
        else:
            product *= (1 + x ** abs(var))
    poly += [coeff * product for coeff in poly]
return poly

def hodge_rank(poly):
    # Placeholder for actual Hodge rank computation
    n = len(poly) - 1
    matrix = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        matrix[i][i-1] = poly[i]
    rank = 0
    for row in matrix:
        if any(row[j] != 0 for j in range(rank)):
            rank += 1
    return rank

def gaussian_elimination(matrix):
    # Perform Gaussian elimination to find the rank of a matrix
    rows, cols = len(matrix), len(matrix[0])
    rank = 0
    for i in range(cols):
        if rank >= rows:
            break
        pivot_row = -1
        for j in range(rank, rows):
            if matrix[j][i] != 0:
                pivot_row = j
                break
        if pivot_row == -1:
            continue
        matrix[pivot_row], matrix[rank] = matrix[rank], matrix[pivot_row]
        for j in range(cols):
            matrix[rank][j] /= matrix[rank][i]
        for j in range(rows):
            if j != rank and matrix[j][i] != 0:
                for k in range(cols):
                    matrix[j][k] -= matrix[j][i] * matrix[rank][k]
        rank += 1
    return rank

n = random.randint(5, 40)
complexity = random.randint(n // 2 + 1, n)
cnf = generate_cnf(n, complexity)
frege_complexity = frege_proof_complexity(cnf)
poly = polynomial_representation(cnf)
h_rank = gaussian_elimination(poly)

metric_name = "Hodge Rank"

```

```

metric_value = h_rank
instances_tested = 1
conjecture_holds = False
counterexample = ""

if frege_complexity > n / 2 and h_rank >= Fraction(math.log(n), 10):
    conjecture_holds = True
elif h_rank <= 2 ** (frege_complexity - n // 2):
    conjecture_holds = False
    counterexample = "Hodge rank too low for given Frege complexity"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Hodge rank too low for given Frege complexity\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad1a94d2.py", line 122, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad1a94d2.py", line 93, in run_trial

```

    poly = polynomial_representation(cnf)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad1a94d2.py", line 41, in polynomial_re
    n = max(abs(var) for var in cnf)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad1a94d2.py", line 41, in <genexpr>
    n = max(abs(var) for var in cnf)
    ^^^^^^^

```

TypeError: bad operand type for abs(): 'tuple'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the required tests to either support or falsify the conjecture. | next: Investigate and fix the error in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12792
2	preregistration	ollama_remote	glm4:latest	0	0	5715
3	novelty	ollama_remote	glm4:latest	0	0	4589
4	novelty	ollama_remote	glm4:latest	0	0	6140
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27980
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11148
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11029
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13372
9	judge	ollama_remote	glm4:latest	0	0	10209

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 102974 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d04434ce6aef.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d04434ce6aef.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d04434ce6aef.tar.gz (if generated)